

系统设计与开发指南



怎样设计与开发一个Java App? (下)

北京理工大学计算机学院
金旭亮

等基本功能探索得差不多了.....

在项目启动阶段，你写了几十个小的Demo，试验了各种App要实现的功能，心中大致有数了，现在就可以正式开始开发你的App了。

这时，要干的第一件事情，就是拿一张白纸，在纸上画出来你的App的“模样”，你需要回答以下问题：

- 你需要确定你的程序包容几个窗体，这些窗体之间如何交换信息；
- 特定的窗体“汇聚”了哪些功能，长成什么样子（即有哪些控件，这些控件如何组合或排列）；
- 用户如何与你的程序交互。比如，用户点击XX按钮，则打开XXX窗体，点击某个Shape，则启动一段动画等等）.....
- 当网络不通时，禁用某些按钮或控件
- 出错时，弹出一个消息，3秒后自动消失
-

系统架构设计



紧接着，确定应用程序架构。

对着已经确定的功能清单，一项一项地问自己：“要完成这个功能，我应该设计哪几个类，这些类如何协作，完成这些功能”，结合原先小Demo得到的成果，确定下要设计哪些类，并且明确这些类的职责（即确定好类中有哪些属性或方法），.....”。

注意不要直接把出于试验目的所编写的Demo中的代码，直接复制粘贴到最终的项目中，应该将这些代码进行重新审视和重构，想清楚整个组合方案，再将其组合到最终的项目中。

注意组合应用多种开发技术设计技术方案



合理应用多线程。如果程序使用到了数据库或需要访问网络或需要完成比较耗时的运算（比如处理高清图像），应该将这些代码移到单独的工作线程中完成。



使用轻量级数据库。如果程序使用到了数据库，推荐使用SQLite这样的嵌入式轻量级数据库，而不要使用MySQL、MS SQL Server这种需要单独安装的数据库。



重用第三方开源库或框架。可以通过互联网找到开源的库或框架，并将它们通过Maven声明集成到自己的程序中，注意在文档中列出你所引用的这些第三方开源项目成果。

推荐的架构设计基本原则



基于职责设计类，相关的类放到相应的包中，包名反映类别信息。每个类都应该有明确的职责（尽量只作一件事），并且应该有明确的归属（比如，与数据库相关的类和代码，都放到db这个包中）

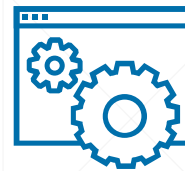


将UI界面与业务功能代码剥离。业务功能代码不要放到窗体中，而应该创建单独的类，并且将它们放到相应的业务包中。UI代码（包括控制器、视图，以及相关的用于处理交互的其他类）可放到ui包中。



应用规范的程序架构。对于JavaFX应用，在表示层推荐基于数据绑定特性，应用MVVM设计模式。在数据存取层，使用Repository封装实现CRUD，在业务层编写XXXService实现业务功能，然后将其组合到控制器中。

应用组件化开发思想提升代码可维护性



应该依据功能和职责将App进行“切分”，划分出若干个组成部分。

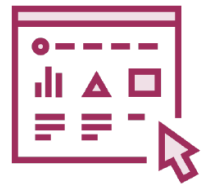


比如，与图像处理相关的代码，可以放到一个名为image_process包中，而与网络操作相关的代码，可以放到另一个名为network的包中。这些包又都归属于特定的模块



采用那些被广泛使用的方案设计你的App架构，比如MVC，Pipeline等等。

UI界面交互设计



使用一个你熟悉的绘图软件，或者就是一叠白纸+铅笔，手工绘出所有App界面，用箭头或各种记号，标识出它们的交互特性（比如点击这个，出现一个动画，然后再导航到另一个界面之类的）。

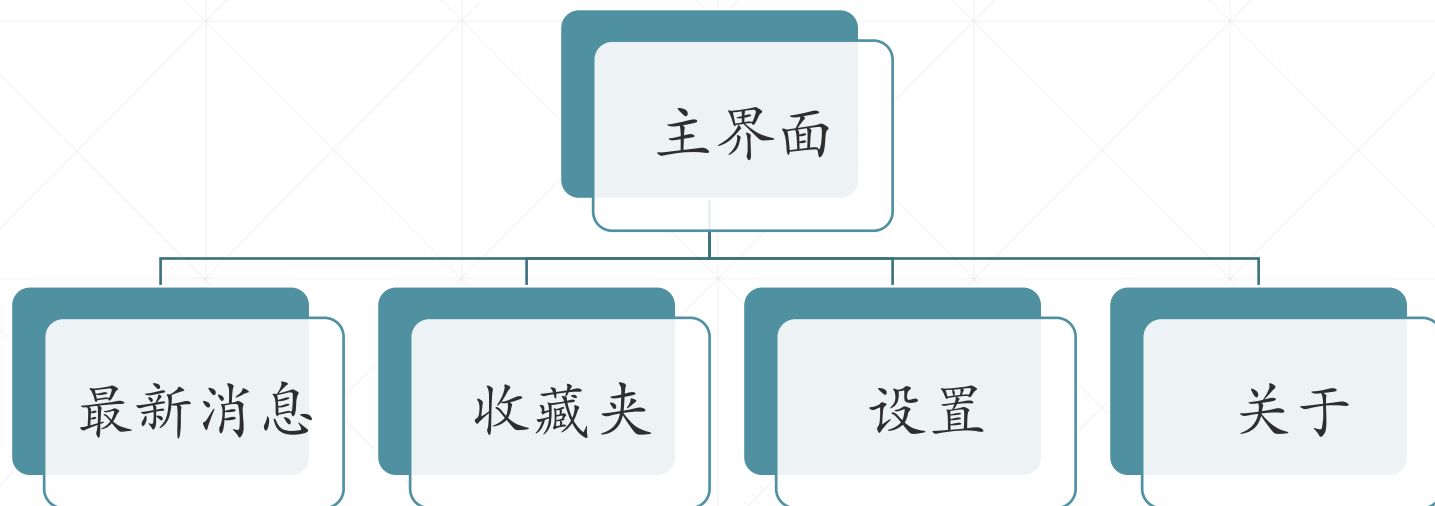


对每一个界面，仔细考虑应该用什么控件去实现它，比如，这里我想实现类似于Excel单元格的功能，Java有没有现成的表格控件库可用？



GitHub是丰富的代码宝库，搜索引擎和AI工具可以快速地帮助你找到可重用的“轮子”。

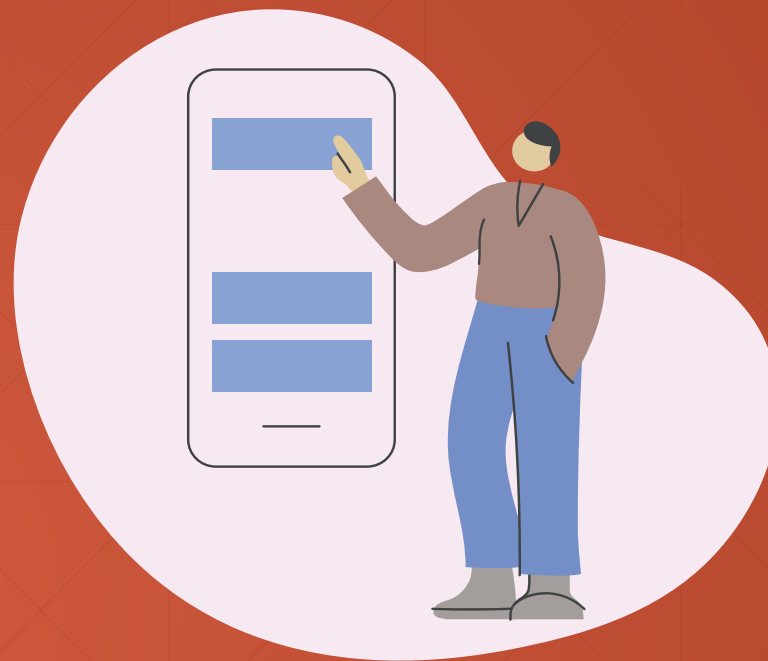
现在你得到了一个App界面总体结构图



使用IntelliJ IDEA创建相应项目，使用JavaFX创建各个窗体，建立起这些窗体之间的关联，之后，再一个窗体一个窗体地实现。

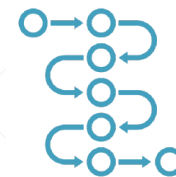


一开始，窗体可以空白的，后头再一个个地实现功能，关键是先把App的整个架子搭好，以后再补充与完善细节。



开发建议与指导

一口口吃饭，一步步做事.....



先实现功能



再保证健壮



最后追求性能和美观。比如，动画功能，是可以最后再加入的.....

源码管理



在开发正式的App时，建议使用一种版本管理工具（推荐使用Git），这样，当你把代码弄得一团糟，App闪退，甚至连编译都过不去时，你可以随时回滚回上一次还能工作的版本。



如果没时间学Git之类版本管理工具，那就可以用最土但也最可靠的方法，每天创建一个压缩包，保存你的项目。压缩包文件名应该包容时间，比如 MySupperApp_2024_11_18.rar。

充分测试



单元测试。 推荐使用JUnit对你的业务功能代码进行单元测试。

功能测试。 程序完成之后，先自己测，然后同学们相互测，要保证你的程序在别人的电脑上也能顺利运行，不出现问题。



在引入新技术或者添加新功能时，不要直接在真实项目中进行测试，而应该创建专用的独立的测试项目，验证此功能可行可用之后，再集成到自己的真实项目中。

最后一步



撰写文档并按时提交。在开发任务结束之后，整理开发中的各种资料，按照教师规定的文档模板撰写开发文档，可以提前提交（不要全部挤到最后截止时间才提交），以避免网络问题导致提交失败。

文档是判分的主要依据，它甚至比程序本身对分数的影响更大，因此，一定要写好文档。

在文档中你一定要尽量地将你最牛的最亮的地方强调并展示出来，说服阅读者觉得你“很棒”，比如，展示出“我拥有百折不挠的毅力，在开发中克服了N多的问题，最终写出了这个牛逼的App”。

不要编造伪造开发经历和夸大所用技术先进性。要知道任课教师敲了二三十年的代码，“编”的东西是很难被糊弄过去的。实事求是！



期待你能开发出让人眼前一亮的优秀App!